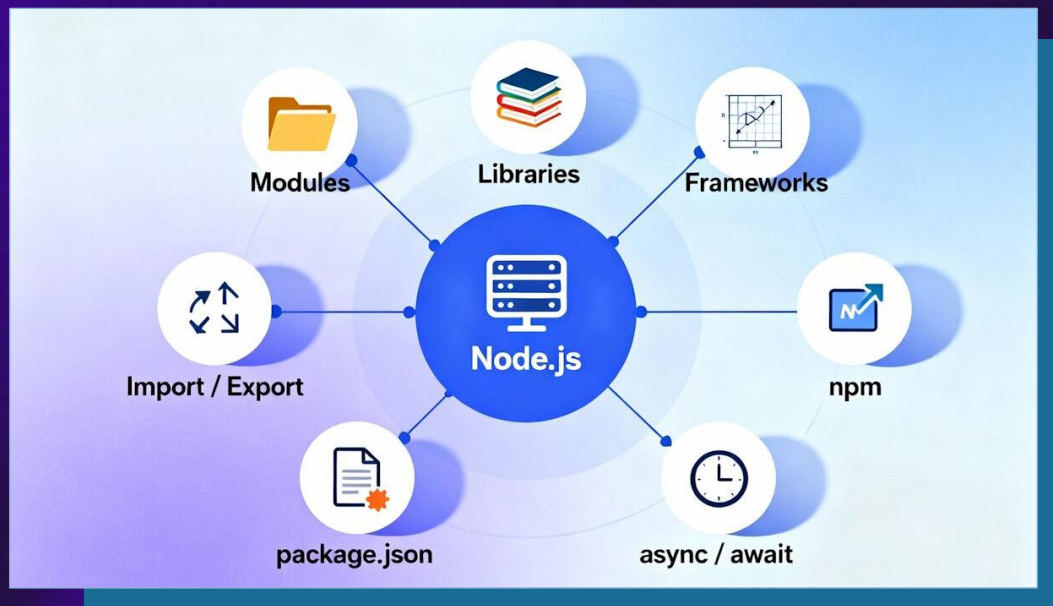


Modules, Libraries, Frameworks

Modules, Import / Export, Node.js, npm, package.json, async / await, Libraries, Frameworks



Svetlin Nakov, PhD

Co-founder @ SoftUni

Agenda

1. Modules, Import / Export

- Importing and exporting objects and types in JavaScript and TypeScript

2. Asynchronous Programming in JavaScript

- Asynchronous execution, promises, async / await

3. Node.js, npm, Project Dependencies

- Node.js projects, package.json, dependencies, npm install, run scripts

4. Using External Libraries in JS

5. Case Study: PDF Invitations App

6. Software Frameworks. Library vs. Framework

7. Case Study: PDF Invoices App



Sli.do Code

#AI-Programming

Join at

slido.com

#AI-Programming



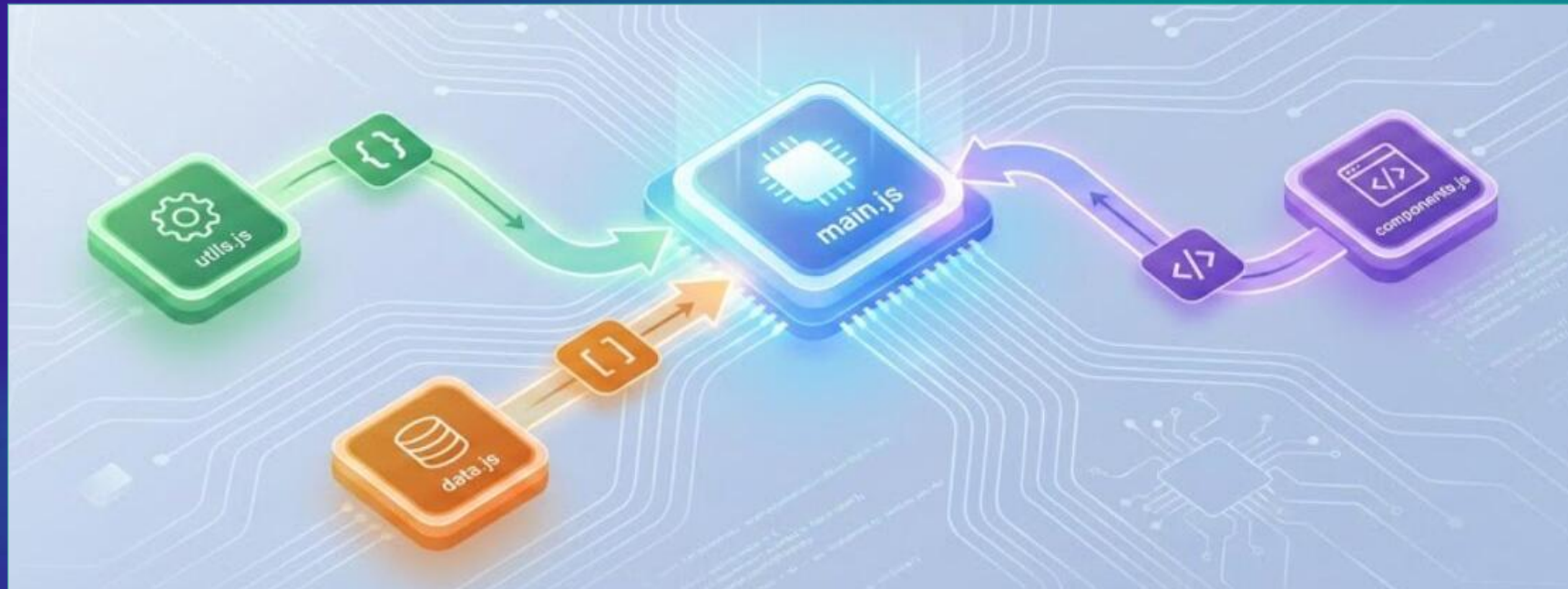
Breaks

20:00 / 21:00



Modules, Import / Export

Importing and Exporting Module Members in JavaScript and TypeScript



JavaScript Modules

- JavaScript **module** is a **.js file**
 - Holding code, designed for reuse across your app
- **JS modules** hold:
 - **Internal** functionality (hidden data and code)
 - **Exported** functionality (public)
- **Private scope** for the entire module content
 - Unless exported, all code and data are **private**
- Exported content can be **imported** and in other modules



Export / Import in JavaScript

- JS modules can **export** named members
- JS modules can **import** the exports from other modules

array-functions.js

```
export function sum(arr) { ... }  
export function min(arr) { ... }  
export function max(arr) { ... }  
export function avg(arr) { ... }  
export const moduleVersion =  
"1.0.0";
```

array-processor.js

```
import { min, max } from  
"./array-functions.js";  
  
let arr = [10, 20, 30, 40];  
console.log("Min", min(arr));  
console.log("Max", max(arr));
```

Importing All Exports

- Modules can **import all named exports** as object

array-aggregations.js

```
import * as Utils from './array-functions.js';  
// Import all "named exports"  
  
const numbers = [10, 20, 30, 40, 50];  
  
console.log("Module version:", Utils.moduleVersion);  
console.log("Sum:", Utils.sum(numbers));  
console.log("Min:", Utils.min(numbers));  
console.log("Max:", Utils.max(numbers));  
console.log("Average:", Utils.avg(numbers));
```


Exporting Members from a Module

- You can **export members** (functions, objects, classes) from a JS module by the keyword **export**

array-utils.js

```
function sum(arr) { ... }  
function min(arr) { ... }  
function max(arr) { ... }  
function avg(arr) { ... }  
export const mVer = "1.0.0"
```

```
const arrUtilsModule = {  
  moduleVersion: mVer,  
  sum, min, max, avg  
};  
export default arrUtilsModule;
```

- The **default export** designates the main exported member

Importing the Default Exports

- Modules can **import** the **default exported** members

array-examples.js

```
import ArrayUtils from './array-utils.js';  
// Import the "default export" member  
  
const numbers = [10, 20, 30, 40, 50];  
  
console.log("Array Utils version:", ArrayUtils.moduleVersion);  
console.log("Sum:", ArrayUtils.sum(numbers));  
console.log("Min:", ArrayUtils.min(numbers));  
console.log("Max:", ArrayUtils.max(numbers));  
console.log("Average:", ArrayUtils.avg(numbers));
```

Export and Import in TypeScript

- In **TypeScript**, we export and import members like in JS
 - During **import**, we use **"./module"**, not **"./module.ts"**

types.ts

```
export type TaskState = "new"  
| "active" | "completed";  
  
export type Task = {  
  title: string;  
  description?: string;  
  state: TaskState;  
};
```

task-manager.ts

```
import { Task, TaskState }  
from "./types";  
  
let task: Task = {  
  title: "Fix dev script"  
  state: "new"  
};  
task.state = "active";
```

CommonJS and Require (Old Style!)

- Older JS codebases, libraries and environments may use **CommonJS** and **require()**, instead of **import**:

old-library.js



```
function sum(arr) { ... }  
function min(arr) { ... }  
function max(arr) { ... }  
  
module.exports = {  
  sum, min, max, avg  
}
```

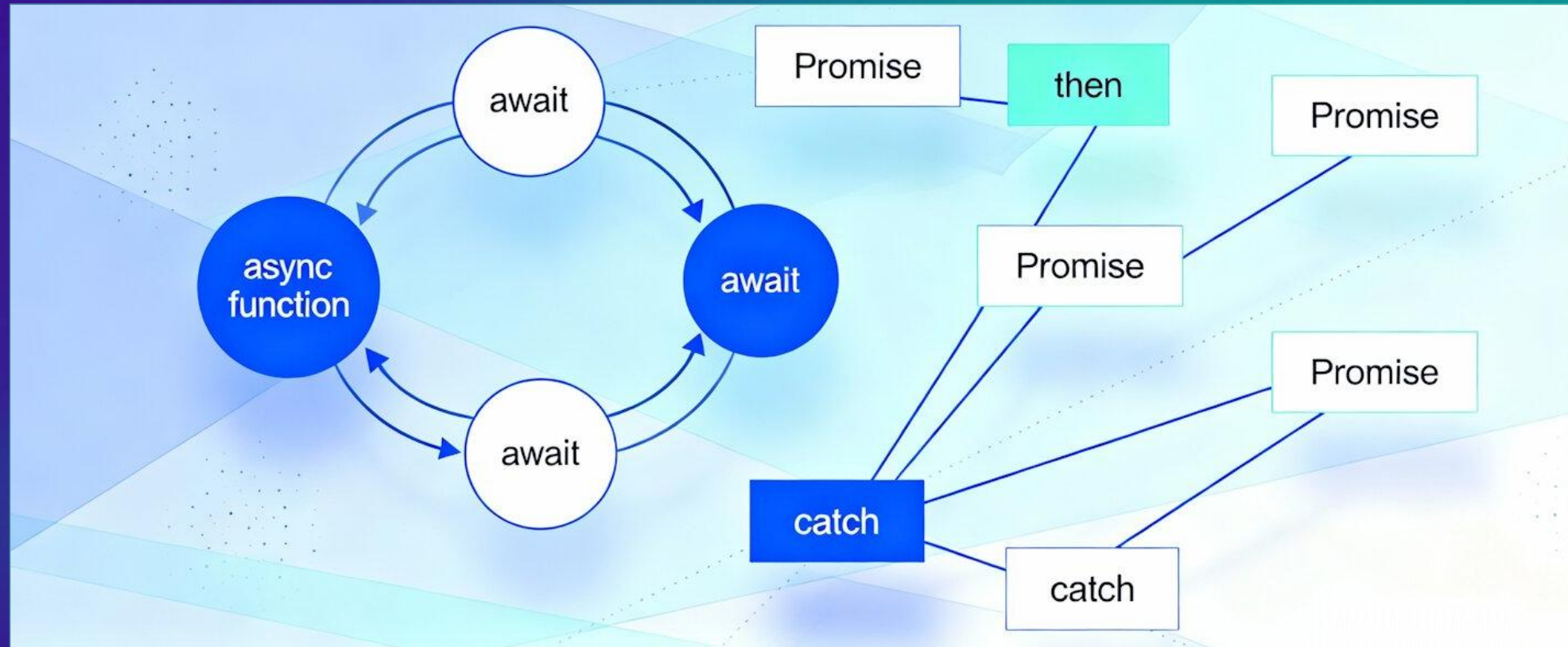
old-client.js



```
let Utils = require('./array-utils.js');  
const numbers = [10, 20, 30, 40, 50];  
  
console.log("Sum:", Utils.sum(numbers));  
console.log("Min:", Utils.min(numbers));  
console.log("Max:", Utils.max(numbers));
```

Asynchronous Programming in JS

Asynchronous Code, Promises, async / await



Synchronous (Blocking) Code

- **Synchronous** code **blocks** until finished

```
console.log("Start...");

function waitSync(durationMs) {
  const end = Date.now() + durationMs;
  do {} while (Date.now() < end); // Highly inefficient!
  console.log("Wait finished.");
}

waitSync(2000); // Blocks for 2 seconds
console.log("End.");
```

```
Start...
Wait finished.
End.
```

Asynchronous (Non-Blocking) Code



- **Asynchronous** code works **in the background** and does not **block** the program execution

```
console.log("Start...");  
  
// The code below will run asynchronously (in the background)  
setTimeout(() => {  
    console.log("Wait finished.");  
}, 2000);  
  
console.log("End.");
```

```
Start...  
End.  
Wait finished.
```

Promises

- A **promise** in JavaScript represents a value that will be available **in the future** (future result)

```
import fs from "fs/promises";  
  
// This starts async file read as a background operation  
// The returned promise represents its eventual result or error  
let fileReadPromise = fs.readFile("data.txt", "utf8");  
  
fileReadPromise  
  .then(data => console.log(data))  
  .catch(err => console.error(err));
```

Promise Chaining

- When you a **value is returned** inside **.then(...)**, it becomes a **new promise** and will be handled in new **.then(...)**

```
fileReadPromise
  .then(data => {
    console.log(data);
    return data.toUpperCase(); // Returns a new Promise
  })
  .then(upper => {
    console.log("Uppercase:", upper);
  })
  .catch(err => console.error(err));
```



Downloading Data from Internet

- Downloading a JSON object from a remote API:

```
fetch("https://jsonplaceholder.typicode.com/todos/1")  
  .then(response => {  
    if (!response.ok)  
      throw new Error(`HTTP error: ${response.status}`);  
    return response.json();  
  })  
  .then(data => console.log("Todo item:", data))  
  .catch(err => console.error(err))
```


Async / Await

- Functions declared **async** automatically return a **promise**

```
async function hello() {  
  return "hello"; // Returns Promise.resolve("hello")  
}  
  
let result = hello(); // result is a promise  
result.then(data => console.log(data));
```

- Use **await** to wait for the call to finish, before go ahead

```
let result = await hello(); // result is a value  
console.log(result);
```

- This waiting is a **non-blocking** operation

Using await with try-catch

```
try {  
  const response = await fetch(  
    "https://jsonplaceholder.typicode.com/todos/1");  
  if (!response.ok)  
    throw new Error(`HTTP error: ${response.status}`);  
  const data = await response.json();  
  console.log("Todo item:", data);  
}  
catch (err) {  
  console.error(err);  
}
```

Node.js Projects and Dependencies

Node.js Projects, package.json, Project Dependencies, npm install, run scripts



Node.js Project Structure

- **Node.js project** == a folder with files
 - JavaScript / TypeScript **source** files
 - E.g. **index.js**, **types.ts**, **data.ts**, etc.
 - Can hold folders: **src/**, **src/data**, ...
 - A **package.json** file
 - The “**project manifest**” (config file)
 - **package-lock.json** – locks the exact package versions
 - A **node_modules** folder (after installing the dependencies)

```
my-app/  
├── package.json  
├── package-lock.json  
├── node_modules/  
└── src/  
    ├── index.js  
    ├── types.ts  
    └── data.ts
```

Sample Node.js Project Structure

- ▶ **my-app/**
 - **package.json**
 - **package-lock.json**
- ▶ **node_modules/**
- ▶ **src/**
 - **index.js**
 - **type.ts**
 - **data.ts**
- ▶ App **root** folder
 - Project **config** file (manifest)
 - Package **versions** lock
- ▶ Installed **libraries** (dependencies)
- ▶ Source **code** files (JS, TS, others)
 - Source code file
 - Source code file
 - Source code file

Project Manifest: **package.json**

- Inside the project manifest **package.json**
 - Package **metadata**, **scripts**, **dependencies**



NPM Scripts

- **Scripts** in **package.json** specify custom terminal **commands**, which you can run for your project
- **Running** a NPM script:

```
npm start
```

```
npm run start
```

```
npm run dev
```

```
"scripts": {  
  "start": "node src/index.js",  
  "dev": "nodemon src/index.js",  
}
```

```
npm run <script-name>
```

- Start the app (the **"start"** script)
- Alternative: run the **"start"** script
- Run the app in **"developer"** mode

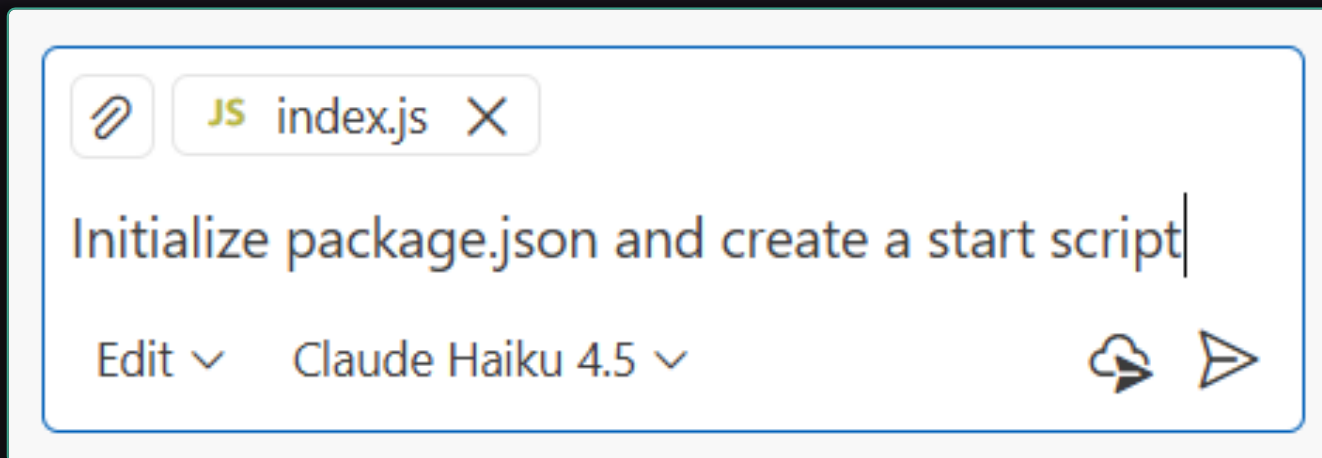
Creating package.json

- Creating a new project manifest `package.json`

```
npm init -y
```

- Alternatively, run a prompt in **GitHub Copilot**:

```
Initialize package.json and create a start script
```



NPM: The Central JavaScript Hub

- **NPM** (Node Package Manager) == software **package registry**
 - Holds 2+ million **reusable JS packages** (libraries)
 - World's **central hub** for JavaScript libraries and tools
 - <https://npmjs.com>
- **NPM** is deeply integrated into the JavaScript ecosystem
 - Node.js projects reference NPM packages through **dependencies** (and **dev dependencies**) in **package.json**
 - **NPM packages** hold JS libraries, components, frameworks and tools, which are installed through **`npm install`**

Node.js Project Dependencies

- Project dependencies in **package.json** are references to other packages (**code libraries**), used by your project

package.json

```
"dependencies": {  
  "express": "^4.21.0"  
},  
  
"devDependencies": {  
  "nodemon": "^3.1.0"  
}
```

- All packages come from **NPM**
- **Runtime dependencies**
 - Required for the app to run
- **Dev dependencies**
 - Needed only during the development

Installing Project Dependencies

- **Installing** Node.js project **dependencies**:

```
npm install
```

- What **`npm install`** it does?

- Reads the project manifest file: **package.json**
- **Downloads** all referenced **dependencies** and **dev dependencies** into a local app folder **node_modules/**
- Creates / updates the **package-lock.json** file

```
PS C:\Projects\my-app> npm install
added 100 packages, and audited 101 packages in 5s
18 packages are looking for funding
  run `npm fund` for details
3 high severity vulnerabilities
To address all issues (including breaking changes), run:
  npm audit fix --force
Run `npm audit` for details.
```

The NPX Tool

- **NPX** (Node Package Execute) **runs npm packages** without installing them (temporary download, run, delete)
- Run the **TypeScript compiler**, without global installation

```
npx tsx
```

- Run a local **Web server** to serve the current folder:

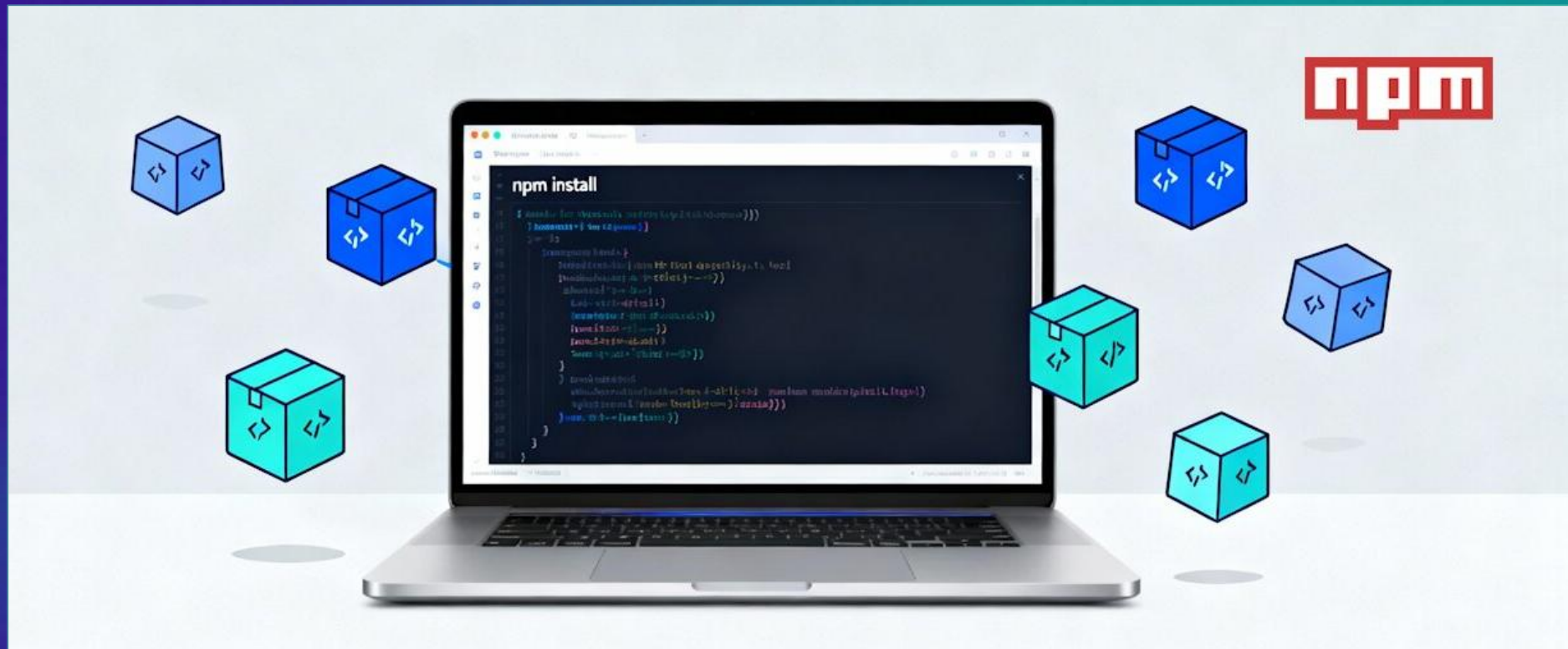
```
npx http-server
```

- List **outdated npm packages** in the current project:

```
npx npm-check-updates
```

External Libraries in JS

Installing and Using External NPM Packages



Installing External Libraries

- **Installing** external **libraries** and **tools** in Node.js:

```
npm install <package-name>
```

- **Example** – installing a library "**uuid**" to generate unique IDs:

```
npm install uuid
```

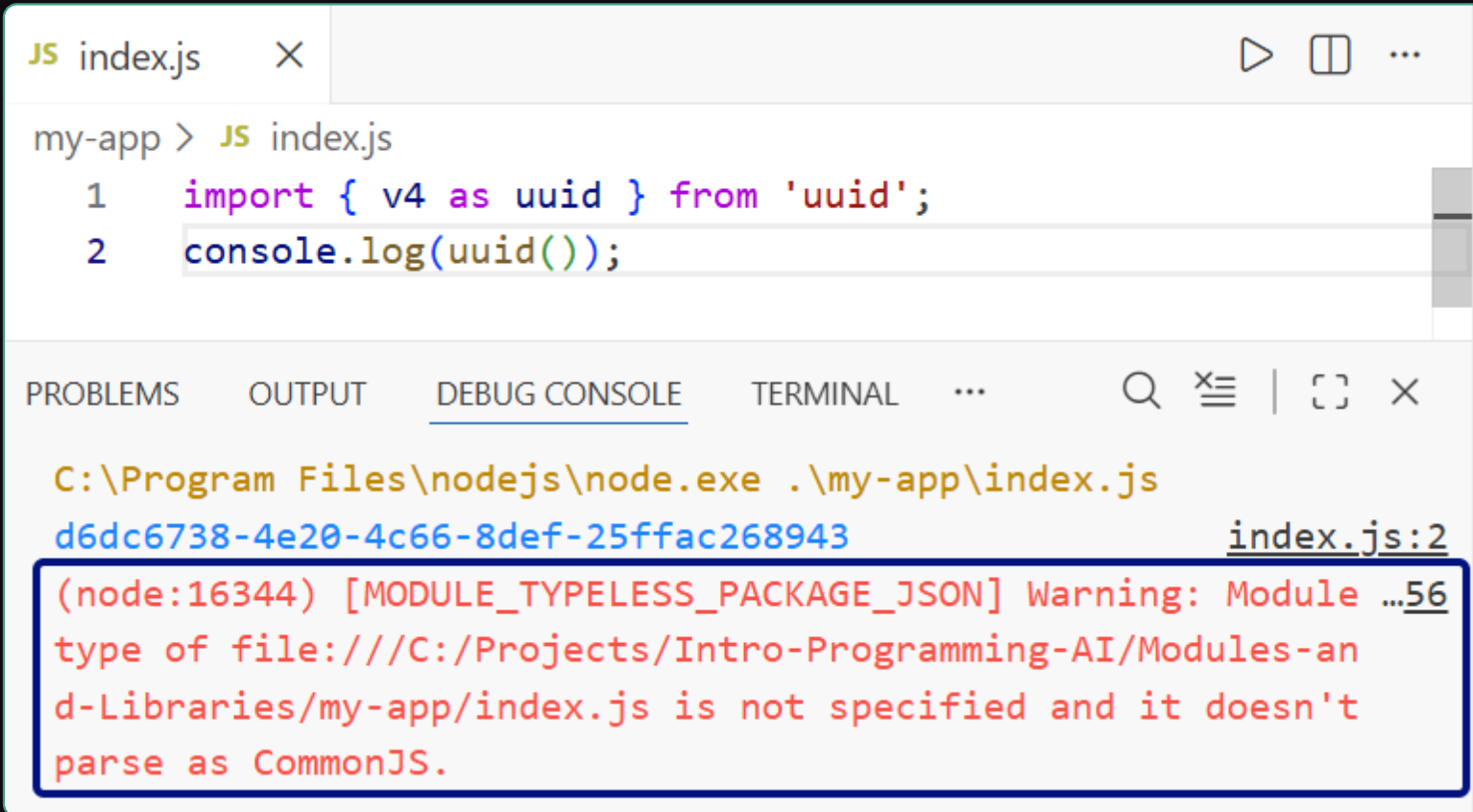
- **Using** the installed library in your JS code:

index.js

```
import { v4 as uuid } from 'uuid';  
console.log(uuid()); // d430368c-c874-4775-99af-68ac930f2c29
```


Running the App

- Run the app with the "uuid" library: `npm start`
 - You will get a **warning**:

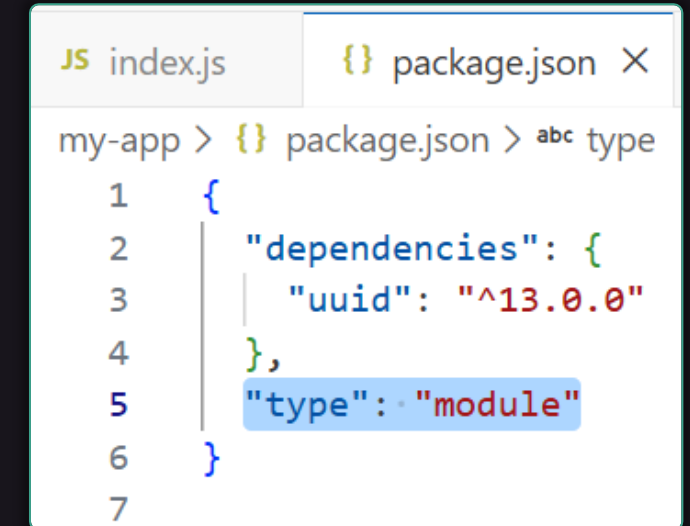


The screenshot shows a VS Code editor with a file named `index.js` containing the following code:

```
1 import { v4 as uuid } from 'uuid';
2 console.log(uuid());
```

The bottom panel shows the **DEBUG CONSOLE** with the following output:

```
C:\Program Files\nodejs\node.exe .\my-app\index.js
d6dc6738-4e20-4c66-8def-25ffac268943 index.js:2
(node:16344) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module ...56
type of file:///C:/Projects/Intro-Programming-AI/Modules-and-Libraries/my-app/index.js is not specified and it doesn't
parse as CommonJS.
```



The screenshot shows a VS Code editor with a file named `package.json` containing the following code:

```
1 {
2   "dependencies": {
3     "uuid": "^13.0.0"
4   },
5   "type": "module"
6 }
7
```

- Fix: change the **package type** to **"module"**

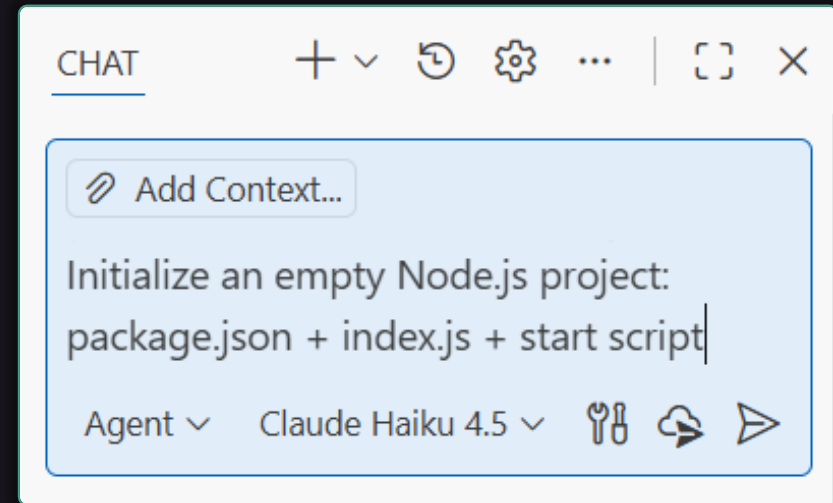
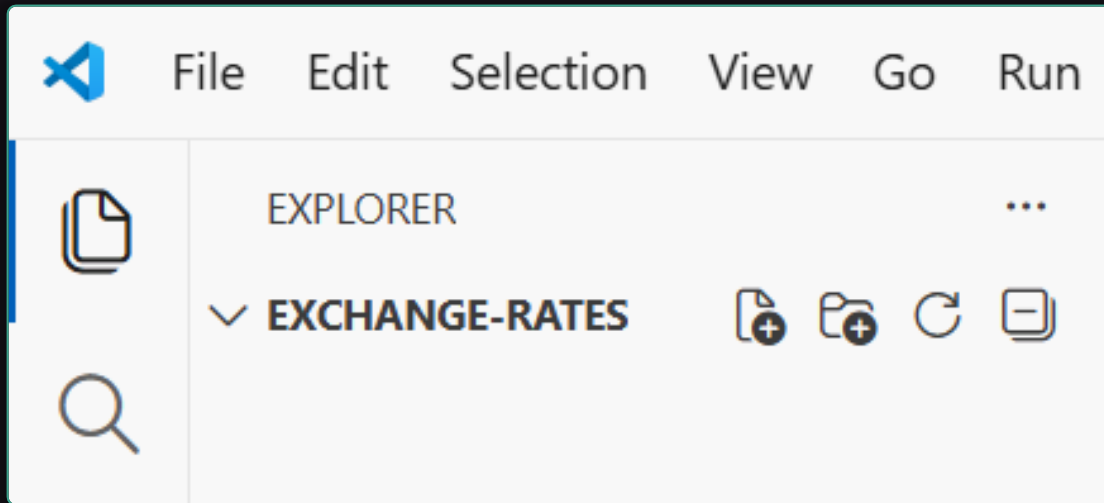
Building an App with NPM (End-to-End)



- Let's build a **Node.js app** with **npm library** (end-to end):
 - **Crypto Rates** app: show major crypto-currency rates
- Steps to building a **Node.js app** with an external **library**:
 1. Create a new **VS Code folder** for your app
 2. Initialize a new **Node.js project**:
package.json + index.js + start script
 3. Install the libraries: "**npm install <package-name>**"
 4. Implement the **app logic**

Building the App: Step by Step

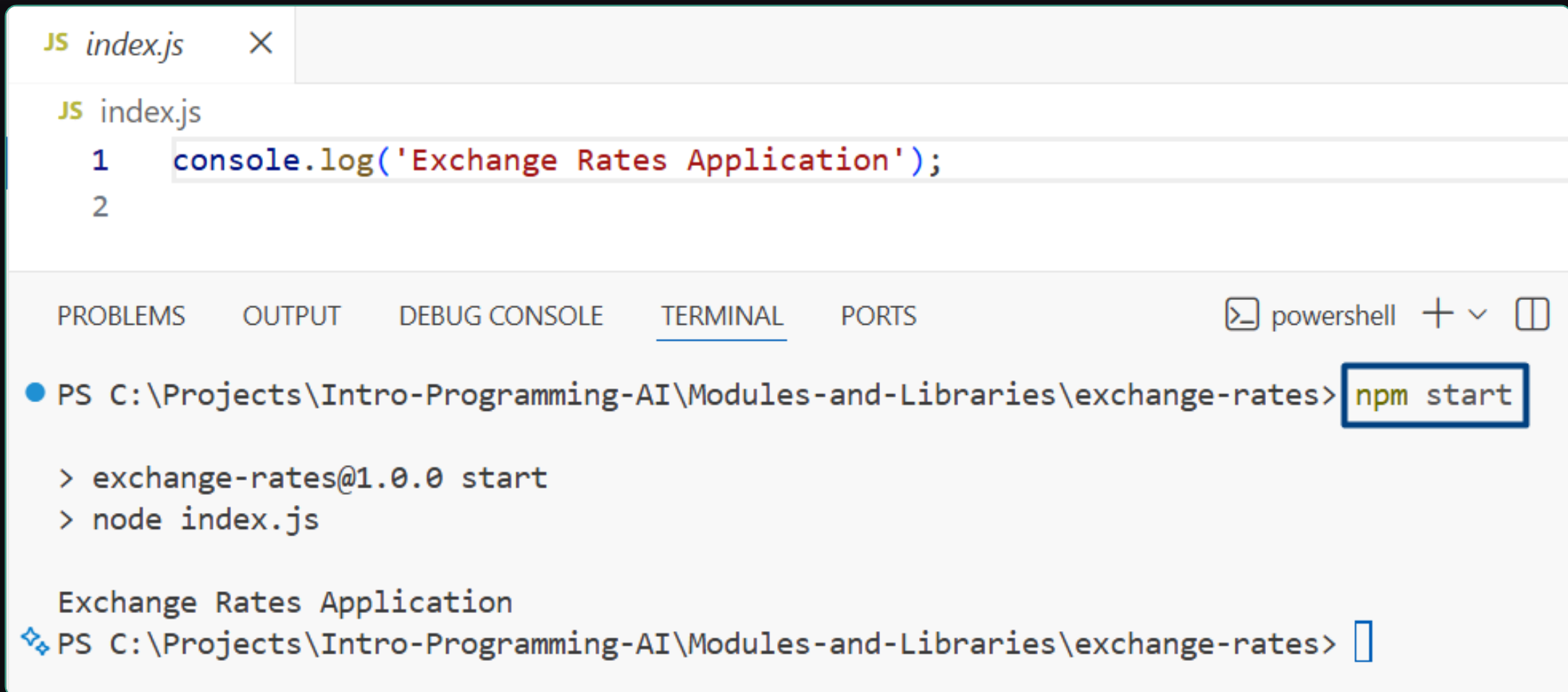
- Create a **new VS Code** folder for your app:
- Run an **AI prompt** in "**Agent mode**":



Initialize an empty **Node.js** project:
package.json + index.js + start script

Run and Test the App

- **Run the app** and test if it will start correctly:



The screenshot shows a Visual Studio Code editor window with a file named `index.js` open. The code in the file is:

```
1 console.log('Exchange Rates Application');
```

Below the editor, the integrated terminal is active, showing the command `npm start` being executed in a PowerShell session. The output of the command is:

```
> exchange-rates@1.0.0 start
> node index.js

Exchange Rates Application
```

The terminal prompt is now `PS C:\Projects\Intro-Programming-AI\Modules-and-Libraries\exchange-rates>`.

Install the Required Libraries

- First, **find** a free crypto-exchange rates **npm package**
 - Use **Internet-enabled AI chatbot** or search in Internet

Find a **npm package** that retrieves exchange rates between major **cryptocurrencies** (such as BTC, ETH, SOL, and BNB) for **free**, without requiring registration, API keys, or any preliminary setup.

SN Find npm package, designed to retrieve the exchange rates between major crypto currencies (like BTC, ETH, SOL, BNB), for free, without registrations, API keys, etc.

I'll search for npm packages that provide free cryptocurrency exchange rates without requiring API keys or registration.

🌐 npm package cryptocurrency exchange rates free no API key

10 results ▾

Let me search more specifically for cryptocurrency-focused packages:

🌐 cryptocompare npm BTC ETH SOL free no API key

10 results ▾

Based on my search results, I found a great npm package that meets your requirements. Here's the best option:

`cryptocompare` - The Best Option

The cryptocompare package works without an API key, though IP-based rate limits apply `npm`. It provides comprehensive cryptocurrency data for BTC, ETH, SOL, BNB, and many others.

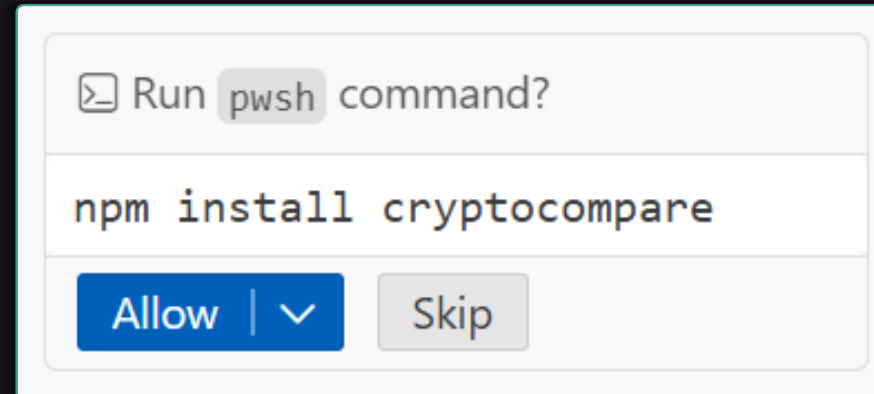
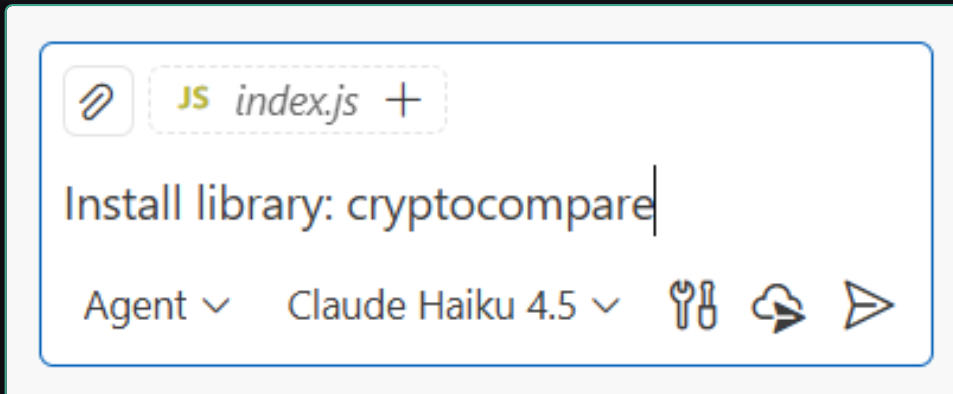
Installing the NPM Library

- **Install** the recommended npm package to your project with a Terminal command:

Terminal window

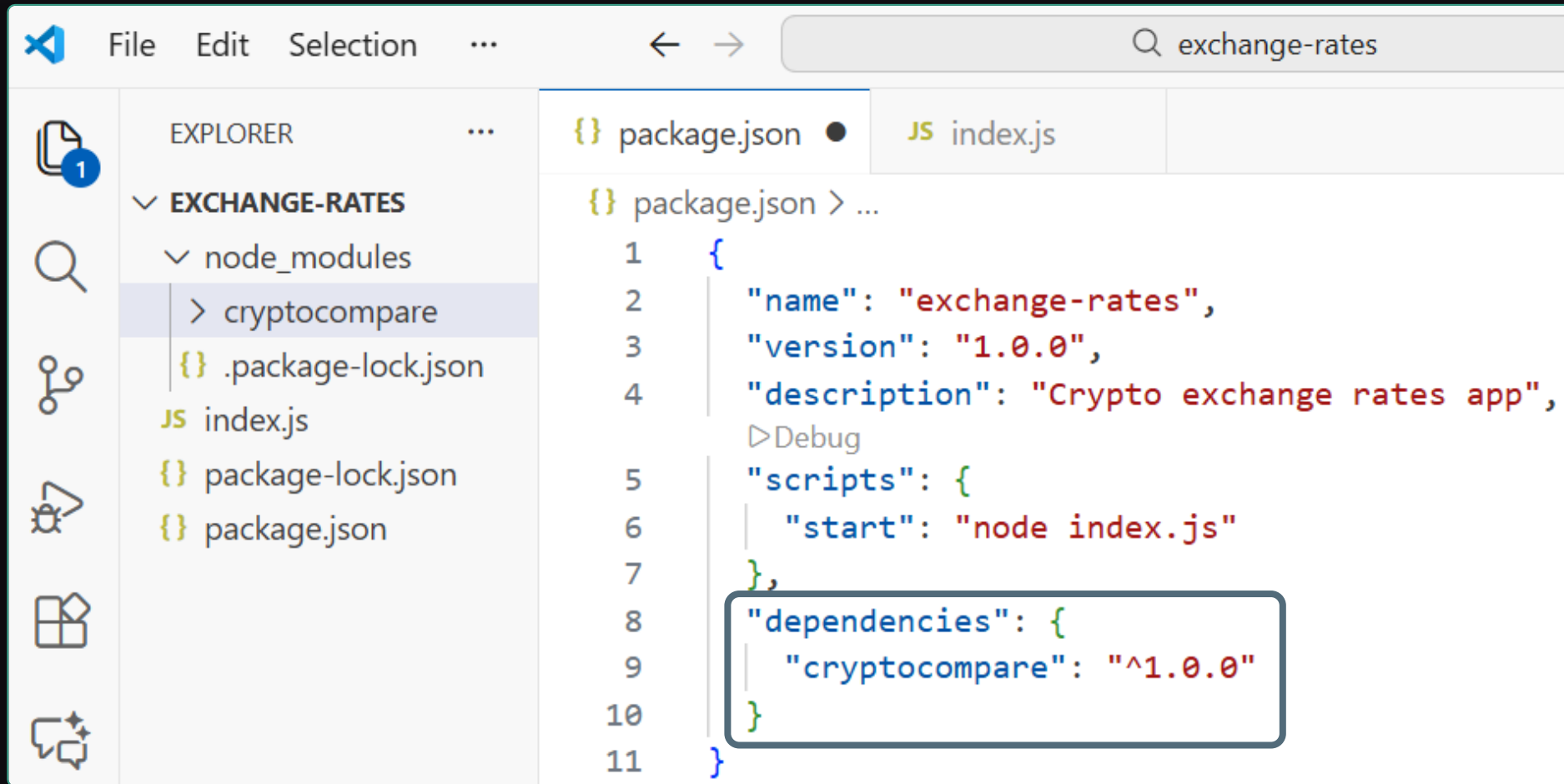
```
npm install cryptocompare
```

- Or ask the **AI dev agent** to install it for you:



The Installed Library

- The npm package **cryptocompare** is now installed:



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays the file structure of a project named 'EXCHANGE-RATES'. Under the 'node_modules' directory, the 'cryptocompare' folder is highlighted. The main editor area shows the 'package.json' file for the project. The 'dependencies' section is highlighted with a red box, showing that 'cryptocompare' is listed as a dependency with the version '^1.0.0'.

```
{} package.json > ...
1  {
2    "name": "exchange-rates",
3    "version": "1.0.0",
4    "description": "Crypto exchange rates app",
5    "scripts": {
6      "start": "node index.js"
7    },
8    "dependencies": {
9      "cryptocompare": "^1.0.0"
10   }
11 }
```


Implement the App Logic

- Now implement the **app logic**, using the installed library, with an AI prompt (in Agent mode):

Implement the **app logic**: retrieve and print on the console the **exchange rates** between USD and all major currencies: BTC, ETH, BNB, SOL, etc.

 **JS** index.js +

Implement the app logic: retrieve and print on the console the exchange rates between USD and all major currencies: BTC, ETH, BNB, SOL, etc.

Agent ▾ Claude Haiku 4.5 ▾   

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

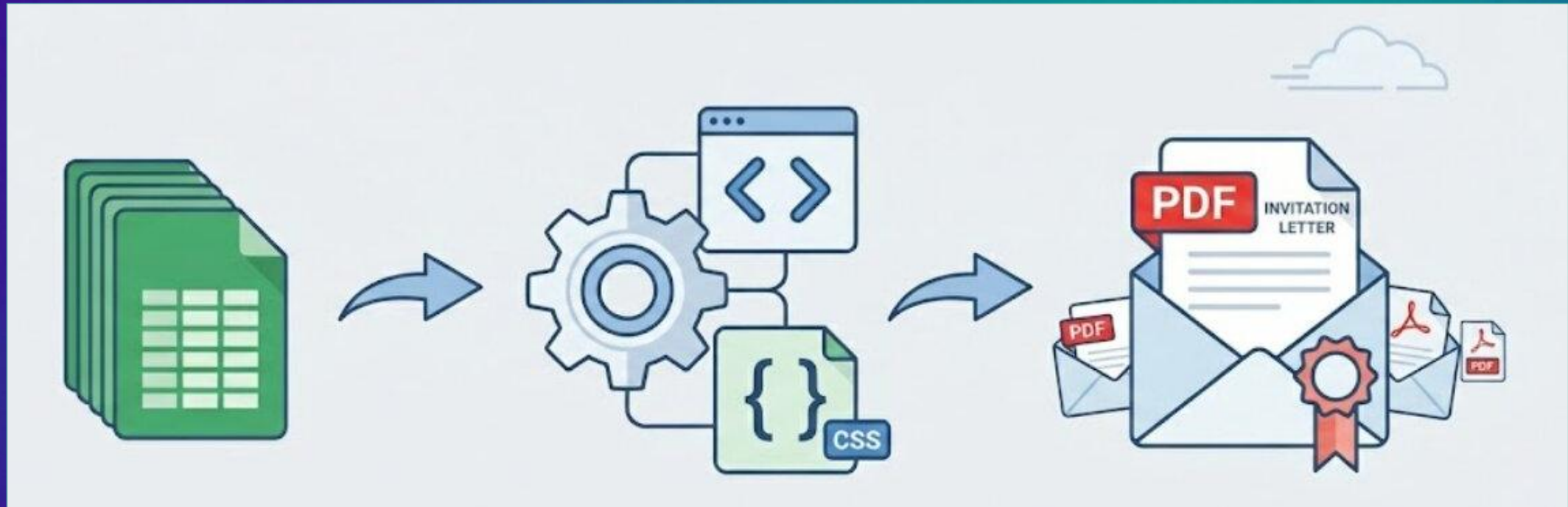
```
PS C:\Projects\Intro-Programming-AI\exchange-rates> npm start
```

Exchange Rates (USD -> Crypto)

```
=====
BTC: $91508.10
ETH: $3039.41
BNB: $897.05
SOL: $138.93
ADA: $0.42
```

PDF Invitations App

Generating **PDF** Invitation Letters
from **Excel** by **HTML + CSS** Template

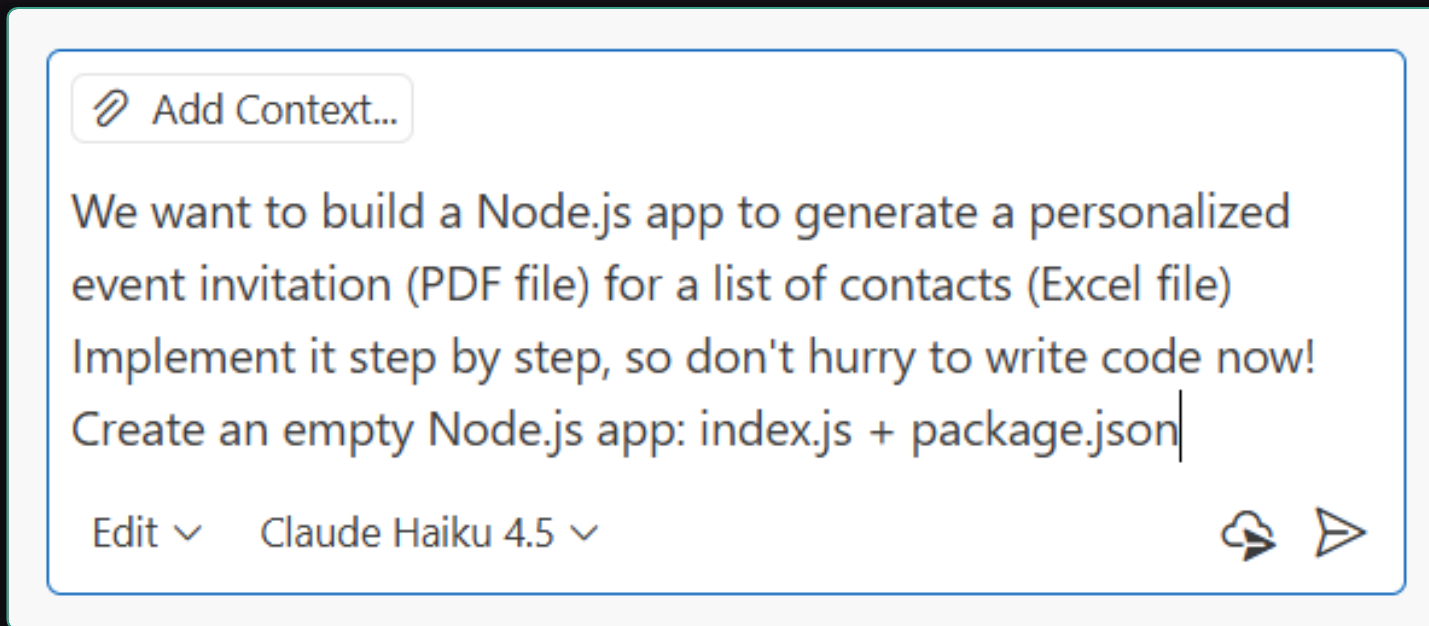
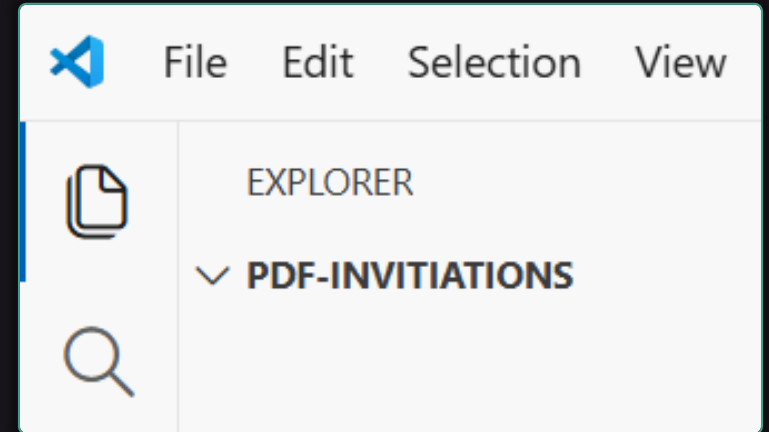


Problem: PDF Invitation

- We want to build a **Node.js app** to generate a personalized **event invitations** (**PDF** files) for a list of contacts (**Excel file**)
 - Implement it **step by step**, so don't hurry to write code now!
 - Create an **empty Node.js app**: **index.js** + **package.json**
 - Install an **Excel reader library** from NPM
 - Install a **HTML-to-PDF library** (render HTML + CSS to PDF)
 - Create a **template** for the invitation as **HTML** + **CSS**
 - Read the **Excel file**. Assume it holds **Person** | **Company** | **Email**
 - For each person, **generate the invitation**, using the template
 - Convert the invitation from **HTML + CSS to PDF**

Creating the App Structure

- Creating the **Node.js app structure**:
 - Create a new **VS Code project** folder
 - Initialize the **Node.js project**



Creating an Empty Node.js Project

package.json

```
{  
  "name": "pdf-invitations",  
  "version": "1.0.0",  
  "scripts": {  
    "start": "node index.js"  
  },  
  "type": "module"  
}
```

index.js

```
console.log(  
  "PDF Invitations app");
```

Terminal window

```
npm run start
```

```
> pdf-invitations@1.0.0 start  
> node index.js  
PDF Invitations app
```

Installing Excel Reader Library

- Installing a **npm library** to read **Excel** files

package.json

```
{  
  ...  
  "dependencies": {  
    "xlsx": "^0.18.5"  
  }  
}
```

Terminal window

```
npm install
```

```
added 9 packages, and audited 10  
packages in 2s
```

```
1 high severity vulnerability
```

```
Some issues need review, and may  
require choosing a different  
dependency.
```

```
Run `npm audit` for details.
```

Installing HTML to PDF Library

- Installing a **npm library** to convert **HTML to PDF**

package.json

```
{  
  ...  
  "dependencies": {  
    "xlsx": "^0.18.5",  
    "puppeteer": "^21.6.0"  
  }  
}
```

CHAT



Install a HTML-to-PDF library (render HTML + CSS to PDF)

 package.json

package.json

Add `puppeteer` as a dependency for rendering HTML/CSS to PDF.

 package.json +2 -1

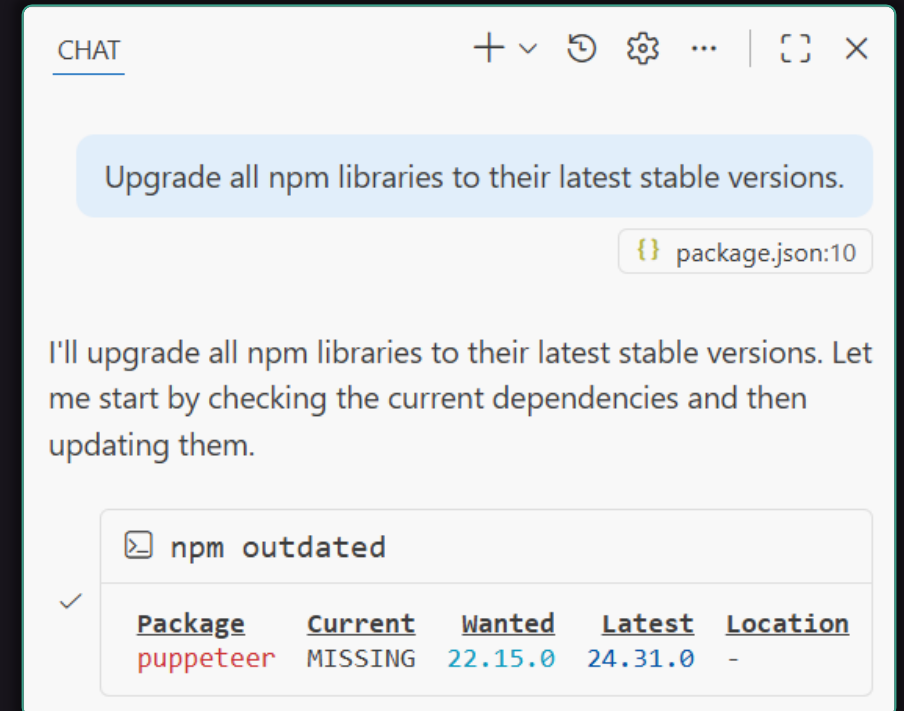
Run `npm install` to install `puppeteer`, which can convert HTML with CSS styling to PDF.

Upgrading the Libraries to Latest

- If you install libraries with AI prompt, it might hard-code certain **library versions** in your **package.json**
 - Typically, **old versions**, which are no longer supported
- **Upgrade** the npm libraries to their **latest versions** with another prompt
 - Run the prompt in **Agent mode**!

Copilot prompt for Agent mode

Upgrade all npm libraries to their latest stable versions.



CHAT

Upgrade all npm libraries to their latest stable versions.

`package.json:10`

I'll upgrade all npm libraries to their latest stable versions. Let me start by checking the current dependencies and then updating them.

npm outdated

Package	Current	Wanted	Latest	Location
puppeteer	MISSING	22.15.0	24.31.0	-

NPM Outdated / NPM Update

- **List outdated** npm packages (minor versions only):

```
npm outdated
```

- **Update** a package to its latest compatible version:

```
npm update <package-name>
```

- **Upgrade all** dependencies to the **absolute latest**:

```
npx npm-check-updates -u  
npm install
```

Creating the HTML Invitation Template

- Sample **AI prompt** to create the **HTML + CSS** invitation letter **template**:

Create a **template** for the invitation as **HTML + CSS**. Person **name** and **company** will come later from **Excel** file.

My company "Acme Technologies" celebrates 10 years in the business. The party will be in "Varna Event Center", 2.12.2025, 19:00.

Be creative. Put **styling** to make the invitation look nice (printed as PDF).

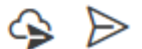
 Add Context...

Create a template for the invitation as HTML + CSS. Person name and company will come later from Excel file.

My company "Acme Technologies" celebrates 10 years in the business. The party will be in "Varna Event Center", 2.12.2025, 19:00.

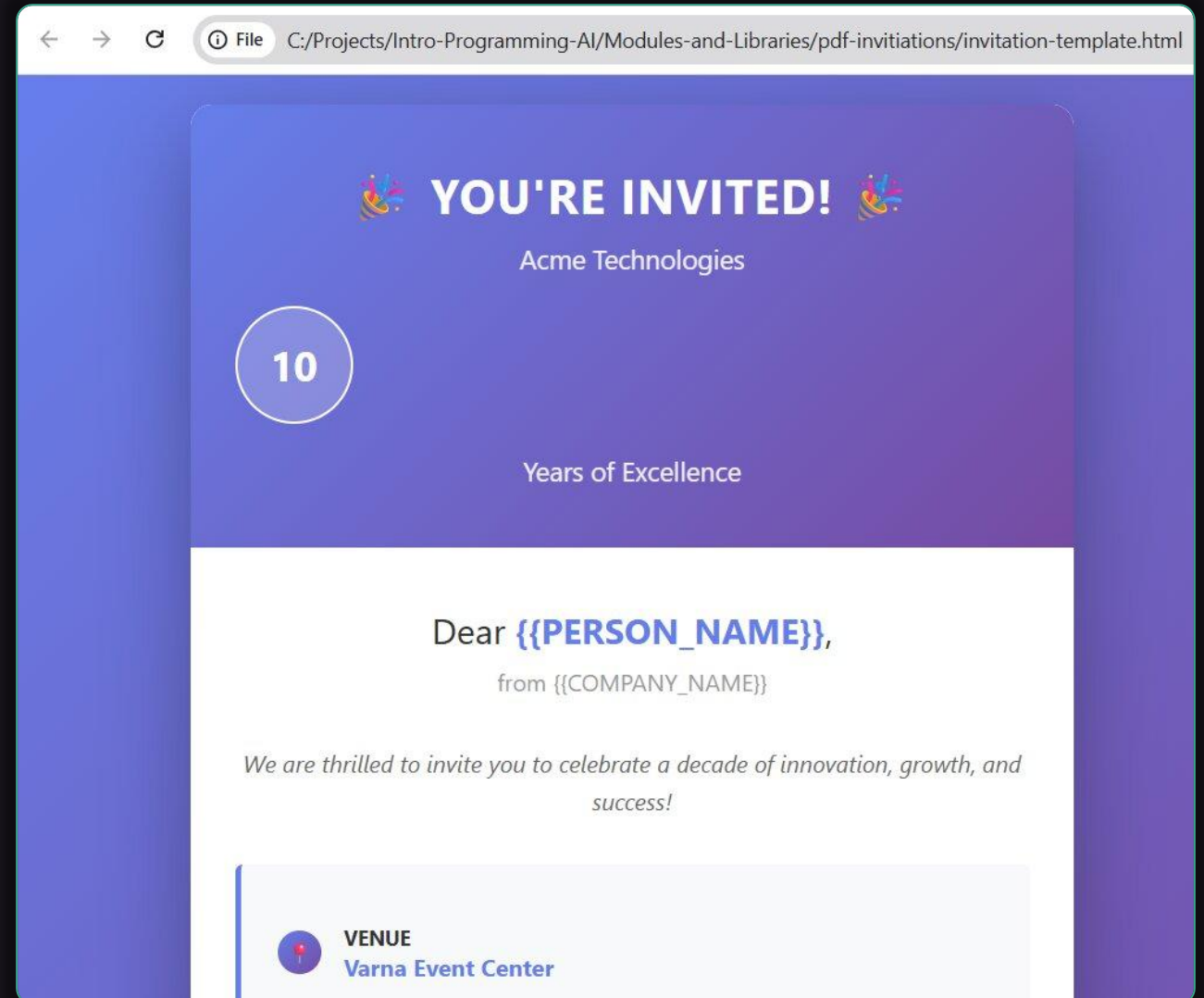
Be creative. Put styling to make the invitation look nice.

Edit ▾ Claude Haiku 4.5 ▾



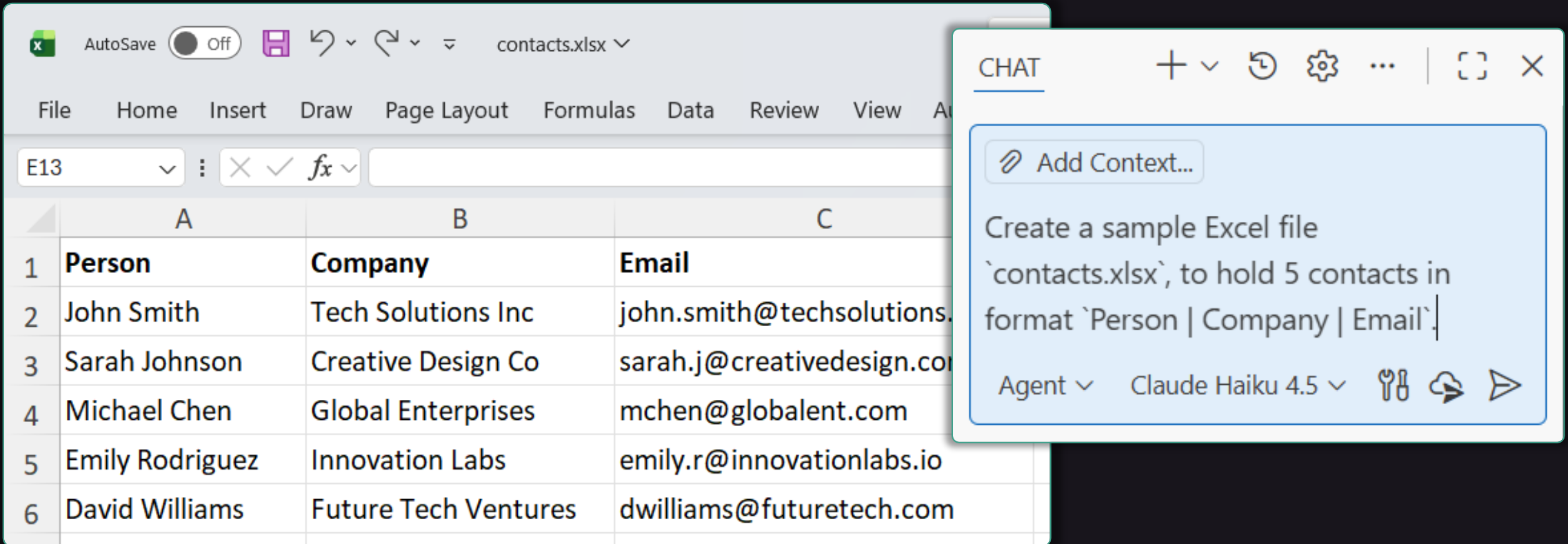
Preview the Invitation Template

- The **HTML template** is easy to edit / customize
- You can generate it in **ChatGPT, Claude, Gemini**
- You can later **edit** it by hand or with prompts
- You can make it "**responsive**" to fill the page width



Creating the Contacts Excel File

- Create the contacts Excel file **`contacts.xlsx`**
 - Either **by hand** (e.g. in Office 365 Excel) or by **AI prompt**



The screenshot displays the Microsoft Excel interface with a file named 'contacts.xlsx'. The 'Home' tab is active, and the formula bar shows 'E13'. A table with 6 rows and 3 columns is visible. The columns are labeled 'Person', 'Company', and 'Email'. The rows contain contact information for John Smith, Sarah Johnson, Michael Chen, Emily Rodriguez, and David Williams. An AI chat window is overlaid on the right side of the Excel window. The chat window has a title bar with a plus sign, a close button, and a refresh button. The chat content includes a prompt to create a sample Excel file and a response from the AI agent.

	A	B	C
1	Person	Company	Email
2	John Smith	Tech Solutions Inc	john.smith@techsolutions.
3	Sarah Johnson	Creative Design Co	sarah.j@creativdesign.com
4	Michael Chen	Global Enterprises	mchen@globalent.com
5	Emily Rodriguez	Innovation Labs	emily.r@innovationlabs.io
6	David Williams	Future Tech Ventures	dwilliams@futuretech.com

CHAT

+ ✕ ↺ ⚙️ ... | [] ✕

🔗 Add Context...

Create a sample Excel file
`contacts.xlsx`, to hold 5 contacts in
format `Person | Company | Email`.

Agent ✕ Claude Haiku 4.5 ✕ 🔑 ☁️ ➤

Implementing the App Logic

- Run an **AI prompt** to implement the main **app logic**:

Now, implement the main **app logic**:

- > Read the **Excel file**. Assume it holds **Person | Company | Email**
- > For each person, generate the invitation, using the template
- > Convert the invitation from HTML + CSS to PDF file

 `JS index.js` +

Now, implement the main app logic:

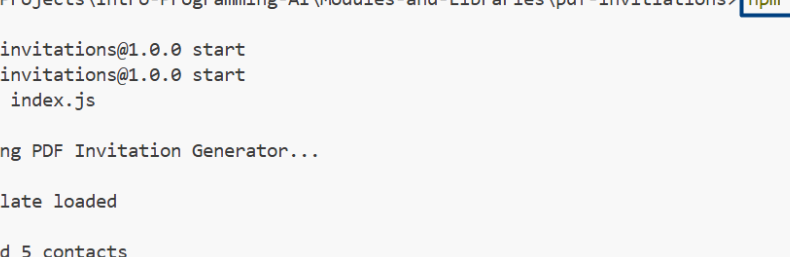
- > Read the Excel file. Assume it holds Person | Company | Email
- > For each person, generate the invitation, using the template
- > Convert the invitation from HTML + CSS to PDF file

Ag... ▾ Claude Haiku ... ▾   

The Result

- invitations

- 📄 invitation_david_williams_5.pdf
- 📄 invitation_emily_rodriguez_4.pdf
- 📄 invitation_john_smith_1.pdf
- 📄 invitation_michael_chen_3.pdf
- 📄 invitation_sarah_johnson_2.pdf



```
PS C:\Projects\Intro-Programming-AI\Modules-and-Libraries\pdf-invitations> npm start

> pdf-invitations@1.0.0 start
> pdf-invitations@1.0.0 start
> node index.js

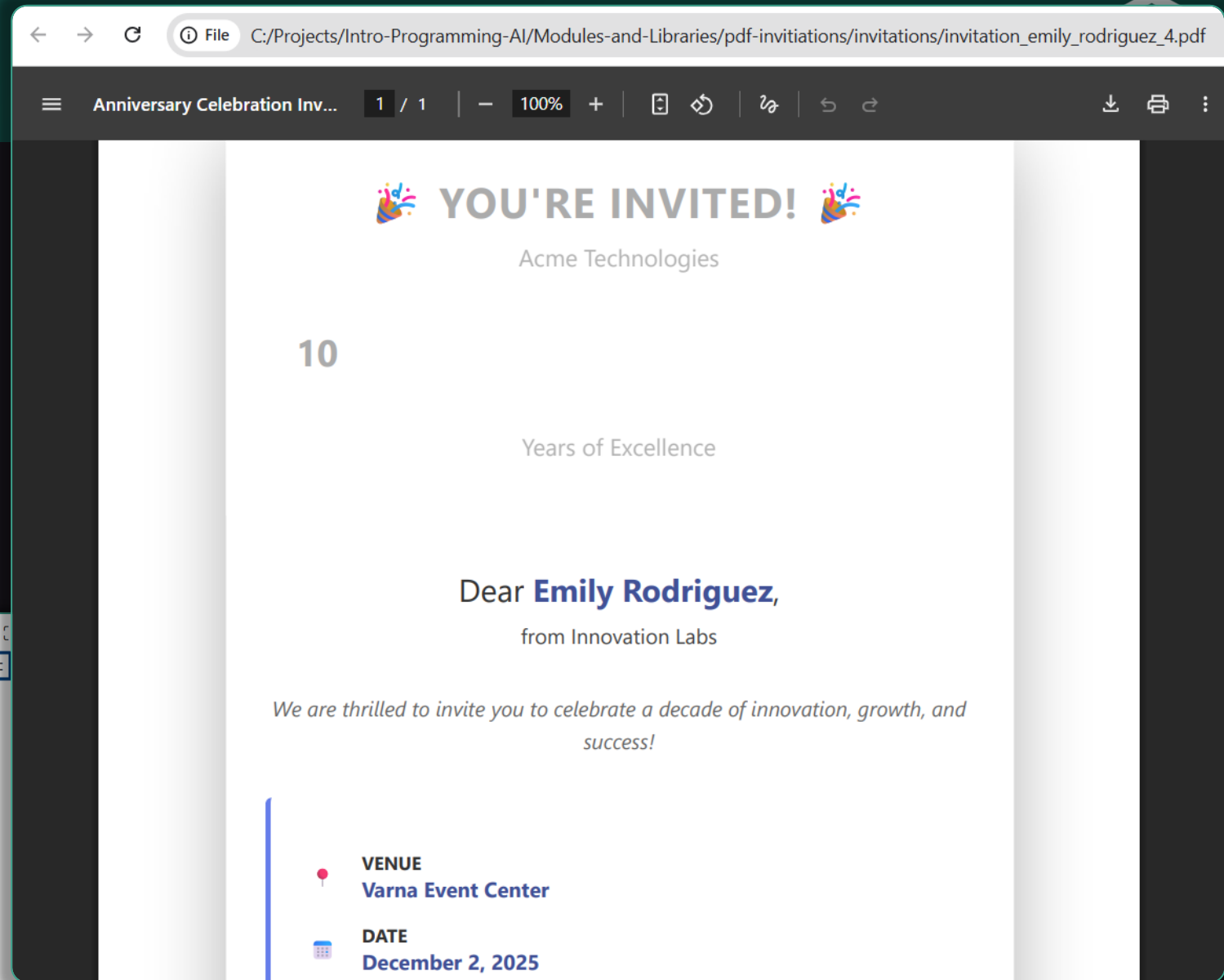
Starting PDF Invitation Generator...

✓ Template loaded

✓ Found 5 contacts

✓ Generated: invitation_john_smith_1.pdf
✓ Generated: invitation_sarah_johnson_2.pdf
✓ Generated: invitation_michael_chen_3.pdf
✓ Generated: invitation_emily_rodriguez_4.pdf
✓ Generated: invitation_david_williams_5.pdf

✓ Invitation generation complete! PDFs saved to: C:\Projects\Intro-Programming-AI\Modules-and-Libraries\pdf-invitations\invitations
PS C:\Projects\Intro-Programming-AI\Modules-and-Libraries\pdf-invitations>
```



Software Frameworks

Software Frameworks. Library vs. Framework

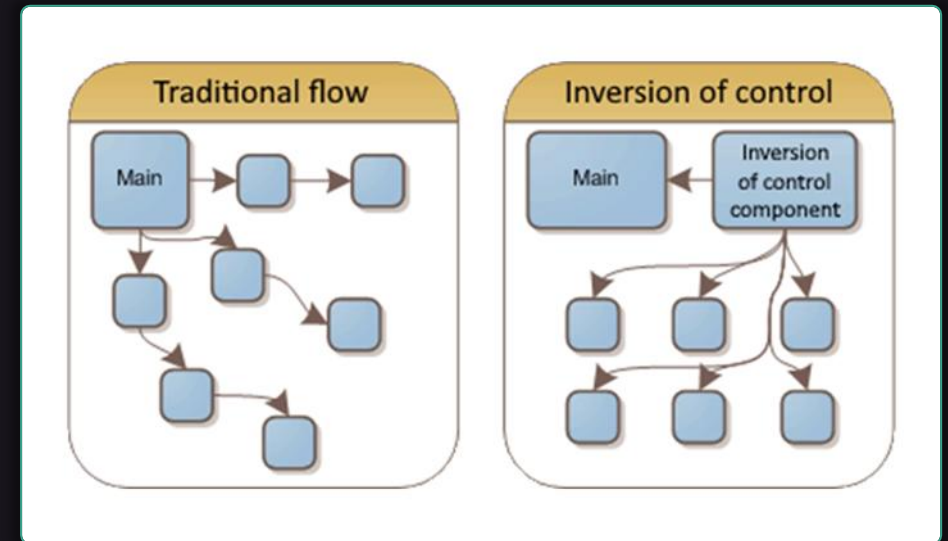


Software Frameworks

- Software **frameworks** are **pre-built foundations** for creating specific types of apps or app functionality
 - Instead of building everything from scratch, you take the **foundations** and extend with app-specific code
 - Frameworks define **app structure**, implement generic functionality, control app flow
- Examples:
 - **Next.js**: a foundation for full-stack apps with React
 - **Express**: a foundation for back-end APIs

Inversion of Control (IoC)

- **Inversion of control (IoC)** principle
 - A function / component / framework does the processing and calls pieces of your code for certain tasks
- Examples of IoC behavior:
 - A **parser** processes a document and calls **events** when it finds certain tokens
 - A **GUI app** manages the UI and calls **events** for the user input



Libraries vs. Frameworks

- **Libraries** provide **components / functionality / UI controls** for integration into existing apps
 - The **app controls the library** components
 - Examples: UI control library, Excel reader
- Development **frameworks** are foundations, which developers extend to build an app
 - The framework **controls the app lifecycle** and your code plugs in (**inversion of control** – IoC)
 - Examples: UI framework, ORM framework



Libraries, Frameworks, Tools

- In the JavaScript ecosystem **NPM** is the hub, holding all JS **libraries, frameworks** and **tools**
- Installing a **library** from **npm**:

```
npm install axios
```

```
npm install uuid
```

- Running a **tool** from npm:

```
npx tsc
```

```
npx npm-check-updates
```

- Creating an app from a **framework**:

```
npx create-next-app@latest
```

Creating a Project by Template

- Creating a JS **project by template**:

```
npm create vite@latest
```

- The above is the same as:

```
npx create-vite@latest
```

- Use **@latest** to ensure the latest app version
 - Not an old one (from the local npm cache)

```
> npm create vite@latest
```

```
> npx
```

```
> create-vite
```

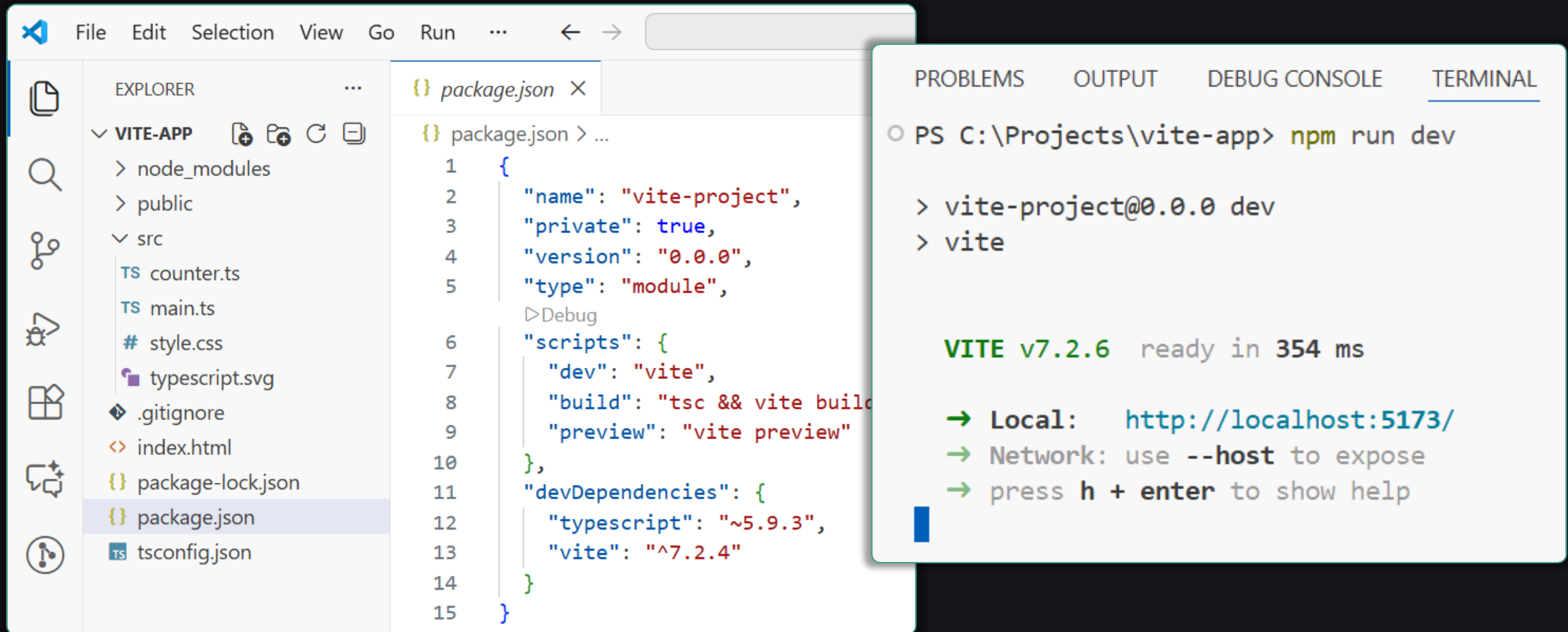
```
|  
◇ Project name:  
vite-app
```

```
|  
◇ Select a framework:  
Vanilla
```

```
|  
◆ Select a variant:  
● TypeScript  
○ JavaScript  
|
```

The Vite + TypeScript Template

- **What's inside** the Vite + TypeScript template?



The screenshot displays the Visual Studio Code interface for a Vite + TypeScript project. The Explorer panel on the left shows the project structure:

- VITE-APP
 - node_modules
 - public
 - src
 - counter.ts
 - main.ts
 - style.css
 - typescript.svg
 - .gitignore
 - index.html
 - package-lock.json
 - package.json
 - tsconfig.json

The package.json file is open in the editor, showing the following configuration:

```
1 {
2   "name": "vite-project",
3   "private": true,
4   "version": "0.0.0",
5   "type": "module",
6   "scripts": {
7     "dev": "vite",
8     "build": "tsc && vite build",
9     "preview": "vite preview"
10  },
11  "devDependencies": {
12    "typescript": "~5.9.3",
13    "vite": "^7.2.4"
14  }
15 }
```

The terminal panel on the right shows the command execution and output:

```
PS C:\Projects\vite-app> npm run dev
> vite-project@0.0.0 dev
> vite

VITE v7.2.6 ready in 354 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```


Creating a Project by Framework

- Example: creating a project, using the **Next.js framework**:

```
npx create-next-app@latest
```

```
npm run dev
```

```
C:\Projects\nextjs-example>npx create-next-app@latest
```

```
Need to install the following packages:
```

```
create-next-app@16.0.6
```

```
Ok to proceed? (y) y
```

```
✓ What is your project named? ... nextjs-app
```

```
✓ Would you like to use the recommended Next.js defaults?
```

```
» Yes, use recommended defaults
```

```
Creating a new Next.js app in C:\Projects\nextjs-example\  
nextjs-app.
```

Generating PDF Invoices

Real-World Example



Problem: PDF Invoices

- We want to build a **front-end app** (Vite + TypeScript) to fill invoice data and print invoices as **PDF** files
 - Implement it **step by step**, so don't hurry to write code now!
 - Create an **empty Vite + TypeScript app**
 - Define the **TypeScript types** to hold invoice data
 - Implement empty app pages with routes: **/**, **/invoices**, ...
 - Implement **"View All Invoices"** in a table
 - Implement **"View Invoice Details"** in a separate page
 - Implement the invoice **PDF printer**

Creating the App Structure

- Create an **empty Vite + TypeScript** app:

```
cd /Projects  
npm create vite@latest pdf-invoices
```

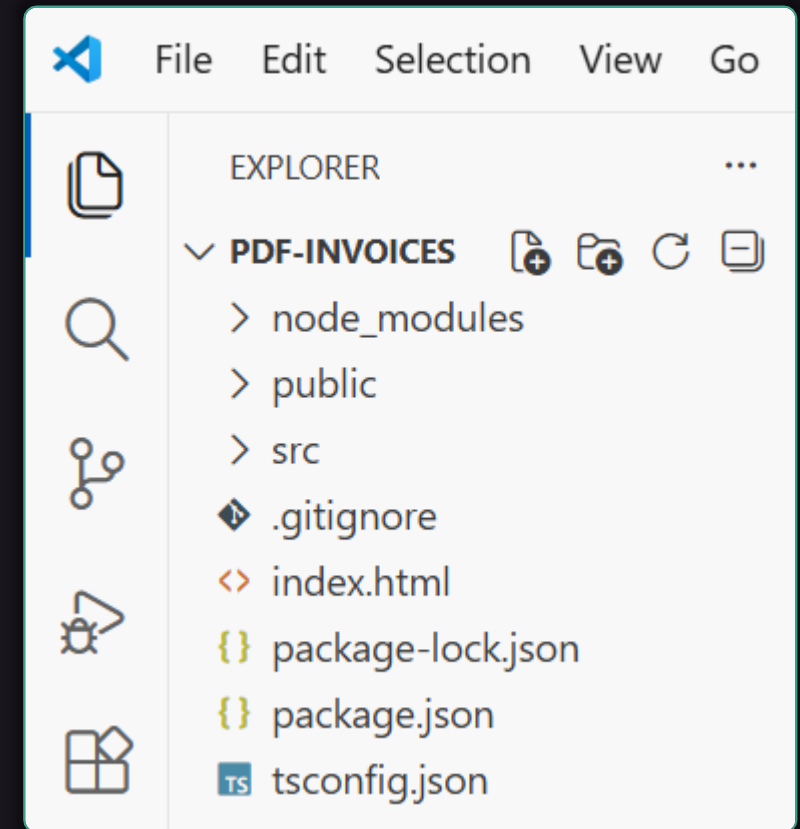
- Open the project in **VS Code**:

```
cd pdf-invoices  
code .
```

- **Install** dependencies and **run** the app:

```
npm install
```

```
npm run dev
```



Invoice Data Types

- Define the **TypeScript types** to hold invoice data:
 - **Invoices** hold the following data: id, date, issuer, recipient, list of items, subtotal amount, VAT amount, total amount
 - **Issuer company** holds the following data: name, id, tax id, address, manager, IBAN
 - **Recipient company** holds the following data: name, id, tax id, address, manager
 - Recipients can be a **company** or a **physical person**
 - **Invoice items** hold the following data: product, quantity, unit (pcs | kg | liters), price, VAT rate, value (without VAT)

Invoices Data Store

- Keep the entire **app data** in a **global data store**
 - Keep it inside a **separate file**
 - Initially, generate **sample invoice data**:
 - 6 invoices with all its details and invoice items
 - The **issuer company** should be the same for the entire app
 - The **VAT rate** should be 20% by default (but still editable)
 - Implement all basic **data operations**: add / edit / delete invoices, add / edit / delete invoice items for an invoice
 - **Keep it simple** and straightforward

Create App Pages

- Implement **empty app pages** with routes:
 - **/** – app home (holds a link to invoices page)
 - **/invoices** – will hold a table of invoices
 - **/invoice/:id/view** – will hold an invoice viewer
 - **/invoice/:id/pdf** – will render an invoice as PDF
- Install appropriate page navigation library
- Implement top **menu navigation**: Home | Invoices
- **Keep it simple** and straightforward

View Invoices

- Implement the **"All Invoices" page**
 - **List** all invoices in a table
 - For each table row add **[View]**, **[PDF]** buttons
 - Buttons should redirect the other pages (will be implemented later)
- Implement the **"View Invoice" page**
 - Display invoice details, well formatted

Implement View / Export as PDF

- Implement "**View as PDF**" for invoices
 - Install a popular **PDF generation library**
 - Render the invoice as **PDF**, well formatted
 - **Display the PDF** inline in the "View Invoice as PDF" page
 - Implement "**Download PDF**" button

Lesson Summary

- JavaScript / TypeScript files (modules) can **export** members (functions, objects, types, etc.)
 - Members can be imported: **import {obj} from "file.js"**
- JS is **asynchronous**: uses promises for long-lasting operations → **promise.then(...).then(...).except(...)**
- Node.js projects hold **package.json + dependencies + scripts + source code**
- JS libraries / tools / frameworks are managed by **npm** & **npx** → **npm install, npm run, npm create, ...**



Questions?



Postbank – Exclusive Partner for SoftUni AI



- One of the leading **banking institutions** in Bulgaria
- Member of the Eurobank Group with € 99.6 billion of assets
- Innovative trendsetter in the digitalization and transformation
- Certified Top Employer 2024 by the international Top Employers Institute
- AI integration for business, learning and development
- AI Assistants for talents: Story Builder, CV Assistant, Interview Trainer
- Proven people care and wellbeing initiatives
- Benefits and unlimited access to professional, personal and leadership trainings and programs
- www.postbank.bg / careers.postbank.bg



Diamond Partners of Software University



**SUPER
HOSTING
.BG**



Diamond Partners of SoftUni Digital



Diamond Partners of SoftUni Digital



HUMAN

IMPULSE MEDIA®



1FOR FIT

NETPEAK
DIGITAL GROWTH PARTNER

**ABC DESIGN &
COMMUNICATION**

Zahara
dig.it.all

ETIENYANEV
Break Your *Limits*. Live Your *Brand*.



Diamond Partners of SoftUni Creative



**SUPER
HOSTING
.BG**



Organization Partners of SoftUni Creative



dequitas.

Фигмаўстор



Studio π

Think Big • Design Smart

дизайнът
на нещата

REDV[®] PRINT
STUDIO

THE
BUCKS
TOWN'S
WORK